Software Requirements Specification (SRS)

A Document Integrity and Confidentiality System

Version: 1.5

Date: July 14, 2025

Author: Okoth Bernard Wycliffe – 23/08325

Project Type: Security and Forensics Undergraduate Project (BISF 2208)

Table of Contents

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Document Integrity and Confidentiality System, designed to protect the confidentiality and integrity of academic documents through cryptographic hashing, and encryption.

1.2 Document Scope

The Document Integrity and Confidentiality System provides a comprehensive solution for protecting and verifying document authenticity, and detecting unauthorized tampering in documents such as certificates, transcripts, and official records.

1.3 Intended Audience

- **Primary Users:** Academic institutions, students, employers
- **Evaluators:** Academic supervisors and external examiners
- **Stakeholders:** Educational institutions requiring document verification

1.4 Product Scope

The system will:

- Generate cryptographic hashes of documents to ensure integrity
- Encrypt hash values using AES-256 to maintain confidentiality of the integrity data
- Embed encrypted hashes into documents using steganography
- Verify document integrity and confidentiality through cryptographic checks
- Provide a user-friendly GUI interface
- Support multiple document formats (PDF, PNG)

1.5 Definitions, Acronyms, and Abbreviations

- **AES:** Advanced Encryption Standard
- **SHA-256:** Secure Hash Algorithm 256-bit

- **LSB:** Least Significant Bit (steganography technique)
- **GUI:** Graphical User Interface
- **API:** Application Programming Interface
- **I/O:** Input/Output operations

---

2. Overall Description

2.1 Product Perspective

This is a standalone desktop application that operates independently of external systems while providing document security features comparable to commercial solutions.

2.1.1 System Context

[User] → [GUI Interface] → [Core System] → [File System]

↓

[Cryptographic Modules] ← → [Steganography Engine]

2.2 Product Functions

The system provides the following major functions:

2.2.1 Document Processing

- Hash generation using SHA-256 algorithm
- AES-256 encryption of hash values
- Document format support (image, PDF)

2.2.2 Steganography Operations

- Text-based steganography for hiding data in documents
- Image-based steganography using LSB method
- Data extraction and recovery

2.2.3 Verification Services

- Document integrity and confidentiality verification
- Tampering detection
- Authentication reporting

2.2.4 User Interface

- Intuitive GUI using Tkinter framework
- File upload/download functionality
- Progress tracking and status reporting

2.3 User Classes and Characteristics

2.3.1 Primary Users

- **Academic Staff:** Moderate technical expertise, requires reliable document protection
- **Students:** Basic technical knowledge, need simple verification process
- **Administrative Personnel:** Limited technical background, require user-friendly interface

2.3.2 Secondary Users

- **IT Support:** High technical expertise, system maintenance and troubleshooting
- **External Verifiers:** Varied technical background, document authentication needs

2.4 Operating Environment

2.4.1 Hardware Platform

- **Minimum:** Intel i3 processor, 4GB RAM, 1GB storage
- **Recommended:** Intel i5 processor, 8GB RAM, 2GB storage
- **Display:** Minimum 1024x768 resolution

2.4.2 Software Platform

- **Operating System:** Windows 10/11 (primary), Linux/macOS (secondary)
- **Python Runtime:** Python 3.8 or higher
- **Development IDE:** PyCharm Community Edition
- **Required Libraries:** cryptography, Pillow, python-docx, PyPDF2

2.5 Design and Implementation Constraints

2.5.1 Technical Constraints

- Must use SHA-256 for hashing (security requirement)
- AES-256 encryption mandatory for data protection
- GUI must be built using Tkinter
- **Maximum file size support:** 100MB per document

2.5.2 Academic Constraints

- 8-week development timeline
- Individual project (no external APIs)
- Must demonstrate all security and forensic concepts
- Code must be well-documented for evaluation

2.6 Assumptions and Dependencies

2.6.1 Assumptions

- Users have basic computer literacy
- Documents are in supported formats
- System runs on machines with adequate resources

- Python runtime environment is properly configured

2.6.2 Dependencies

- Python cryptography library for encryption
- Pillow library for image processing
- PyPDF2 library for pdf document processing
- Tkinter for GUI (included with Python)
- Standard Python libraries for file operations

---

3. System Features

3.1 Document Hash Generation

3.1.1 Description

Generate SHA-256 cryptographic hash of uploaded documents to create unique fingerprints.

3.1.2 Functional Requirements

- **REQ-3.1.1:** System shall generate SHA-256 hash for any uploaded document
- **REQ-3.1.2:** Hash generation shall complete within 5 seconds for files up to 10MB
- **REQ-3.1.3:** System shall display hash value in hexadecimal format
- **REQ-3.1.4:** Hash values shall be stored temporarily in memory during processing

3.1.3 Inputs

- Document file (PDF, PNG)
- User selection to generate hash

3.1.4 Outputs

- 64-character hexadecimal hash string
- Confirmation message
- Processing status indicator

3.2 Hash Encryption

3.2.1 Description

Encrypt generated hash values using AES-256 encryption to protect integrity data.

3.2.2 Functional Requirements

- **REQ-3.2.1:** System shall encrypt hash values using AES-256 algorithm
- **REQ-3.2.2:** System shall generate unique encryption keys for each session
- **REQ-3.2.3:** Encrypted data shall be encoded in base64 format
- **REQ-3.2.4:** System shall provide key management functionality

3.2.3 Inputs

- SHA-256 hash value
- Encryption key (password or provided)

3.2.4 Outputs

- Encrypted hash data
- Encryption key (for storage/sharing)
- Encryption status confirmation

3.3 Steganography Implementation

3.3.1 Description

Hide encrypted hash data within documents using steganographic techniques.

3.3.2 Functional Requirements

- **REQ-3.3.1:** System shall implement text steganography for text documents
- **REQ-3.3.2:** System shall implement LSB steganography for image files
- **REQ-3.3.3:** Hidden data shall not visibly alter original document
- **REQ-3.3.4:** System shall maintain document format integrity

3.3.3 Text Steganography

- **Input:** Text document, encrypted hash
- **Process:** Embed data using whitespace or character manipulation
- **Output:** Visually identical document with hidden data

3.3.4 Image Steganography

- **Input:** Image file, encrypted hash
- **Process:** Modify least significant bits (LSB) of pixel values
- **Output:** Visually identical image with embedded data

3.4 Document Verification

3.4.1 Description

Extract hidden data and verify document integrity through hash comparison.

3.4.2 Functional Requirements

- **REQ-3.4.3:** System shall generate fresh hash of current document
- **REQ-3.4.4:** System shall compare hashes and report verification status

3.4.3 Verification Process

1. Extract encrypted hash from document
2. Generate current document hash
3. Compare protected and current hashes
4. Report verification result

3.5 File Management

3.5.1 Description

Handle file upload, processing, and output operations.

3.5.2 Functional Requirements

- **REQ-3.5.1:** System shall support file upload via GUI dialog
- **REQ-3.5.2:** System shall validate file format before processing
- **REQ-3.5.3:** System shall save processed files to designated output directory
- **REQ-3.5.4:** System shall maintain original file properties where possible

3.6 User Interface

3.6.1 Description

Provide intuitive graphical interface for all system operations.

3.6.2 Functional Requirements

- **REQ-3.6.1:** System shall provide main window with menu navigation
- **REQ-3.6.2:** System shall display progress indicators for long operations
- **REQ-3.6.3:** System shall show clear error messages and success notifications
- **REQ-3.6.4:** System shall support file drag-and-drop functionality (future integration)

---

4. External Interface Requirements

4.1 User Interfaces

4.1.1 Main Window

- **Size:** 800x600 pixels minimum
- **Layout:** Menu bar, toolbar, main content area, status bar
- **Navigation:** Tab-based interface for different functions
- **Responsive:** Adapts to window resizing

4.1.2 Dialog Windows

- **File Selection:** Standard OS file picker
- **Settings:** Configuration options dialog
- **About:** Application information and credits
- **Help:** User manual and tutorials

4.2 Hardware Interfaces

4.2.1 Storage Interface

- Read/write access to local file system
- Temporary file creation capabilities
- Directory structure management

4.2.2 Memory Interface

- RAM utilization for file processing
- Temporary data storage during operations
- Memory cleanup after processing

4.3 Software Interfaces

4.3.1 Operating System Interface

- File system API calls
- Process management
- System clipboard access

4.3.2 Python Library Interfaces

- **Cryptography:** Encryption/decryption operations
- **Pillow:** Image processing and manipulation
- **PyPDF2:** PDF processing and manipulation
- **Tkinter:** GUI framework components
- **Hashlib:** Hash generation functions

4.4 Communication Interfaces

- No network communication required
- All operations performed locally
- File-based data exchange only

---

5. Non-Functional Requirements

5.1 Performance Requirements

5.1.1 Response Time

- **Hash generation:** Maximum 5 seconds for 10MB files
- **Encryption/Decryption:** Maximum 2 seconds
- **GUI responsiveness:** Maximum 100ms for user interactions
- **File loading:** Maximum 10 seconds for large files

5.1.2 Throughput

- Support concurrent operations where applicable
- Batch processing capability for multiple files
- Efficient memory usage during processing

5.2 Security Requirements

5.2.1 Data Protection

- **REQ-5.2.1:** All sensitive data shall be encrypted using AES-256
- **REQ-5.2.2:** Encryption keys shall be properly generated and managed

- **REQ-5.2.3:** Temporary files shall be securely deleted after use
- **REQ-5.2.4:** System shall not store unencrypted sensitive data

5.2.2 Algorithm Requirements

- **REQ-5.2.5:** SHA-256 hashing algorithm mandatory
- **REQ-5.2.6:** AES-256 encryption in CBC mode
- **REQ-5.2.7:** Secure random number generation for keys
- **REQ-5.2.8:** Cryptographic libraries must be industry-standard

5.3 Reliability Requirements

5.3.1 Availability

- **REQ-5.3.1:** System shall have 99% uptime during operation
- **REQ-5.3.2:** Graceful handling of unexpected shutdowns
- **REQ-5.3.3:** Automatic recovery from minor errors

5.3.2 Error Handling

- **REQ-5.3.4:** Comprehensive error logging
- **REQ-5.3.5:** User-friendly error messages
- **REQ-5.3.6:** Graceful degradation on failures

5.4 Usability Requirements

5.4.1 Ease of Use

- **REQ-5.4.1:** Intuitive interface requiring minimal training
- **REQ-5.4.2:** Clear visual feedback for all operations
- **REQ-5.4.3:** Consistent UI elements and behavior
- **REQ-5.4.4:** Helpful tooltips and guidance

5.4.2 Accessibility

- **REQ-5.4.5:** Keyboard navigation support
- **REQ-5.4.6:** High contrast mode compatibility
- **REQ-5.4.7:** Screen reader friendly interface

5.5 Scalability Requirements

- Support for files up to 100MB
- Efficient processing of batch operations (future integration)
- Modular architecture for future enhancements
- Configurable performance parameters

5.6 Portability Requirements

- **Primary:** Windows 10/11 compatibility
- **Secondary:** Linux and macOS support
- Standard Python libraries only
- Minimal system dependencies

6. Other Requirements

6.1 Academic Requirements

6.1.1 Educational Objectives

- Demonstrate understanding of cryptographic concepts
- Show practical application of security and forensic principles
- Implement software engineering best practices
- Document all system design decisions and trade-offs

6.1.2 Deliverable Requirements

- A fully working system
- Comprehensive unit test suite
- Technical documentation and user manual
- Live demonstration of all features

6.2 Legal and Regulatory Requirements

6.2.1 Intellectual Property

- Use only open-source libraries and frameworks
- Ensure all code is original or properly attributed
- Comply with integrity and confidentiality policies
- Respect software licenses and terms of use

6.2.2 Data Privacy

- No personal data collection or storage
- Local processing only (no cloud services)
- User consent for all operations
- Secure disposal of temporary data

6.3 Quality Assurance Requirements

6.3.1 Testing Requirements

- Unit tests for all core functions
- Integration tests for system workflows
- User acceptance testing scenarios
- Performance benchmarking

6.3.2 Documentation Requirements

- Code documentation
- Technical architecture documentation
- User manual
- Installation and setup guide

6.4 Maintenance and Support

6.4.1 Maintenance Requirements

- Modular code structure for easy updates
- Version control using Git
- Change log documentation
- Bug tracking and resolution process

6.4.2 Support Requirements

- User manual and FAQ documentation
- Error message explanations
- Troubleshooting guide

Appendices

Appendix A: Glossary

- **Document Integrity:** Assurance that a document has not been altered
- **Steganography:** Practice of concealing information within other non-secret data
- **Hash Function:** Mathematical algorithm that maps data to fixed-size values
- **Encryption:** Process of encoding information to prevent unauthorized access

Appendix B: References

- NIST Cryptographic Standards and Guidelines
- Python Cryptography Documentation
- Academic Project Guidelines
- Software Engineering Best Practices

Appendix C: Revision History

Version	Date	Author	Description
1.0	June 29, 2025	Okoth Bernard Wycliffe	Initial SRS document

---

Document Status: Draft

Next Review Date: To be determined

Approval Required: Dr. Paul Abuonji (Academic Supervisor)

X

Paul Abuonji
Dr.